

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a deep blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene, digital aesthetic.

DL for ENG

LECTURE 4.2

Deep Neural Networks

Several hidden layers: what depth buys, what it costs, how much capacity a problem needs, and how to tell when a model has too much.

Remus Teodorescu

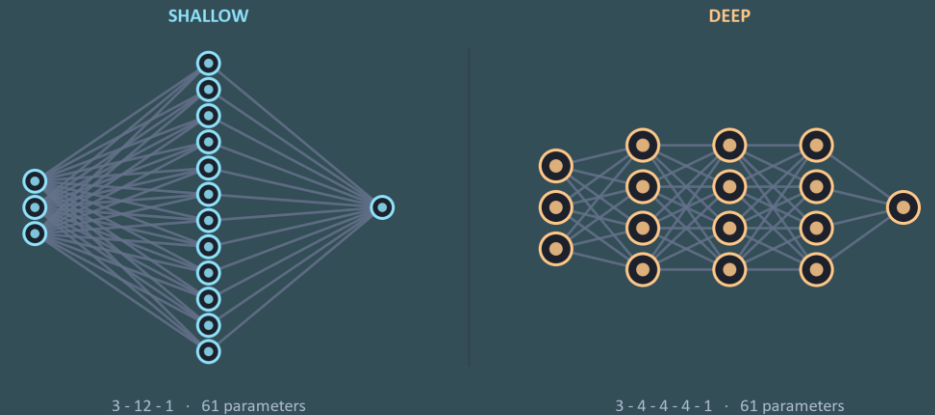
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

depth · capacity · overfitting · regularisation

SHALLOW AND DEEP, 61 PARAMETERS



Same parameter count. The deep one can represent more functions, and that difference is the first half of this lecture.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Deep networks** — composition, layer notation, the parameter count
- 2 **Depth and width** — what depth buys, and why it was hard to train
- 3 **Capacity** — underfitting, overfitting, data splits, bias and variance
- 4 **Regularisation** — weight decay, early stopping, and the rest
- 5 **Choosing capacity** — a procedure, with and without held-out data
- 6 **Challenges** — what goes wrong first, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

training	gradients and backpropagation — L6.1
very deep nets	residual connections, normalisation — L5.1
physics in the loss	the residual as a regulariser — L7
structured inputs	convolution and graphs — L5.1

As in the last lecture, every curve is already fitted. The parameters are found by an optimiser that stays a black box until lecture six.

Recap

THE TEN QUESTIONS OF L4.1 — FROM PERCEPTRON TO NEURAL NETWORK

- 1** What is a neuron? Write its equation and name each term in it. *UDL ch. 3.1*
- 2** What separates a perceptron from a neuron, and why does that difference matter for training? *GBC ch. 1.2 and 6.1*
- 3** Why can a single neuron not represent XOR, and what does one hidden layer of two neurons change? *Ex_04 notebook 01; GBC ch. 6.1*
- 4** What does a ReLU network with D hidden neurons compute, exactly, and how many linear regions does it have? *UDL ch. 3.2; Ex_04 notebook 03*
- 5** Why does a network without activation functions reduce to a linear model, whatever its depth? *UDL ch. 3.1*
- 6** Which activation would you choose for a regression problem, and which when second derivatives enter the loss? Why? *UDL ch. 3.1*
- 7** What does the universal approximation theorem state, and what are two things it does not tell you? *UDL ch. 3.2; Cybenko (1989)*
- 8** How many parameters has a network with D_i inputs, one hidden layer of D neurons and D_o outputs? Give the formula. *UDL ch. 3.3 and 3.4*
- 9** Why do several outputs of one physical state share one hidden layer? *UDL ch. 3.4*
- 10** How would you choose the width and depth of a first network for a new problem, and what result would make you change them? *UDL ch. 4.5; Ex_04 notebook 04*

Composition

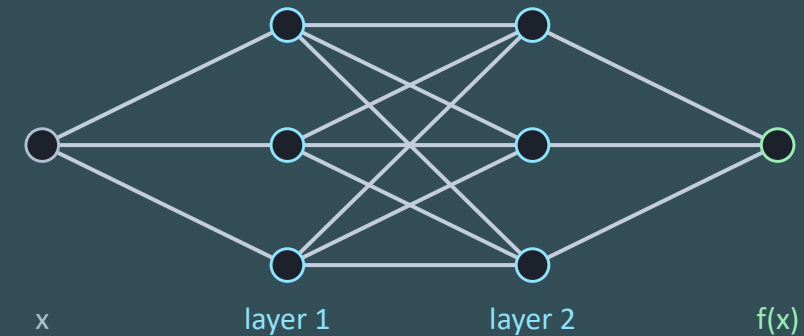
ONE NETWORK FED INTO THE NEXT

- **a deep network** — the output of one shallow network is the input of the next
- **each layer computes a representation** — the next layer works in it
- **nothing new** — the same neuron, the same activation, the same loss
- **K layers** — K compositions of the one-layer function

$$f(x) = f_K \circ f_{K-1} \circ \dots \circ f_1(x)$$

f_k the k th layer, K how many layers, $\circ$ means feed the output onward

TWO SHALLOW NETWORKS, JOINED



WHAT A LAYER DOES

A layer transforms the space; the next layer solves an easier problem in the transformed space. The hidden layer that solved XOR did this once. Depth does it K times.

Layer Notation

WEIGHTS AS A MATRIX, BIASES AS A VECTOR

- **one layer** — a weight matrix W , a bias vector b , and the activation applied elementwise
- **the output of layer k** — is the input of layer $k + 1$
- **the same arithmetic as before** — written to fit on one line
- **in PyTorch** — `nn.Linear` holds W and b ; the activation is a separate module

$$\mathbf{h}_k = a(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k)$$

$\mathbf{h}_k$ the output of layer k , $\mathbf{W}_k$ its weight matrix, $\mathbf{b}_k$ its bias vector, a the activation

ONE LAYER, AS CODE

`W (D × D)` the weight matrix

`b (D)` the bias vector

`h = a(W h + b)` one layer, entire

`nn.Linear(D, D)` PyTorch: W and b together

`nn.ReLU()` the activation, separately

Parameter Count

WIDTH COSTS D^2 , DEPTH COSTS D^2 PER LAYER

- **a weight matrix between two layers of width D** — holds D^2 numbers
- **doubling the width** — quadruples the parameters; doubling the depth doubles them
- **print the count at the start of every run** — one line, and it catches the commonest configuration mistake

$$P = D_i D + (K - 1) D^2 + D D_o + (K D + D_o)$$

P parameters, D_i inputs, D the width, K hidden layers, D_o outputs

SOME REAL COUNTS

$1 \times 10 \times 1$	31 — the shallow net of L4.1
$2 \times 32 \times 32 \times 1$	1,185 — a typical PINN
4 layers of 50	about 8,000 — a flow field
ResNet-50	25 million
a 2020 language model	175 billion

The first three are what this course builds. The last two are on the slide for the distance: a heat equation does not need a data centre.

Linear Regions

REGIONS MULTIPLY WITH DEPTH

- **one layer of D ReLU neurons** — $D + 1$ linear regions, as in the last lecture
- **a second layer** — folds the first layer's output, so the regions multiply
- **K layers of D neurons** — regions grow roughly as D to the power K
- **at equal parameter count** — depth represents more; that is the case for depth

theory

depth can need exponentially fewer neurons (adversarial functions)

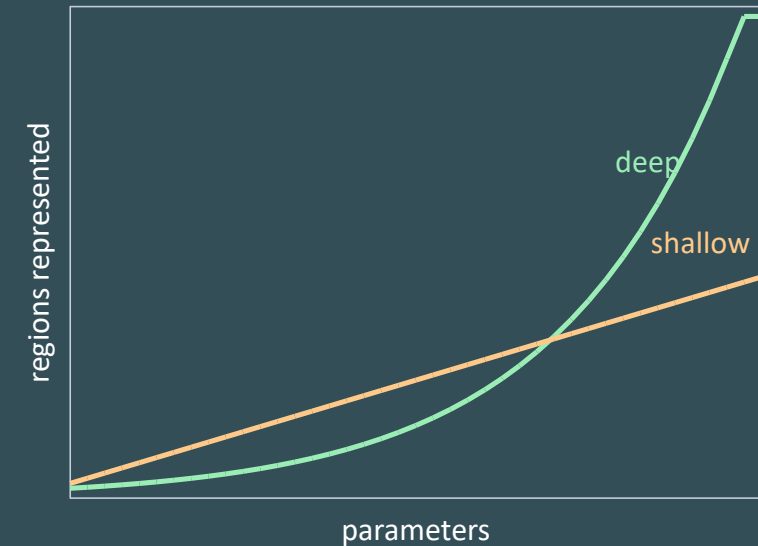
practice

4 to 8 layers of 20 to 50 neurons solves every problem in this course

start

4×32 , and change it only on evidence — notebook 04

REGIONS AGAINST PARAMETERS



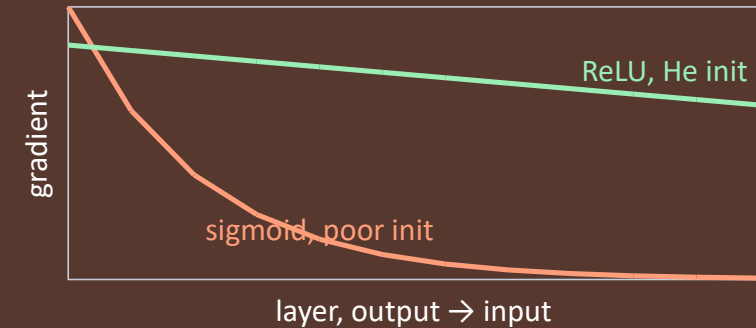
No units on the axes; the claim is the ordering. Notebook 04 measures it at a fixed parameter budget, spent on width, on depth, and in between.

Training Difficulties

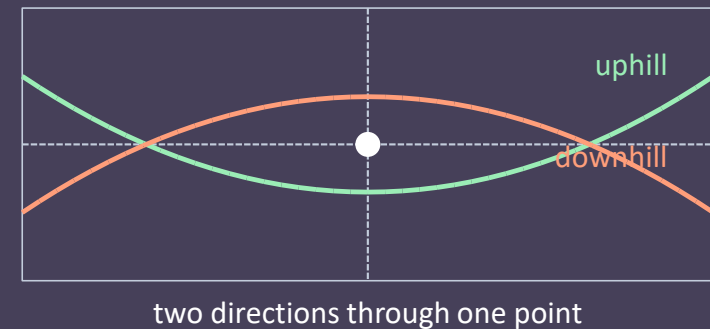
TWO REASONS DEPTH WAS HARD, AND WHAT CHANGED

- **vanishing gradient** — a long product of derivatives below one leaves the early layers almost no gradient
- **the fix** — ReLU, He initialisation (2015); for very deep networks residual connections and normalisation
- **many minima** — in high dimensions most stationary points are saddles, and descent escapes a saddle
- **the minima it finds** — give similar loss in practice; why is only partly understood

GRADIENT SIZE BY LAYER



A SADDLE, NOT A BOWL

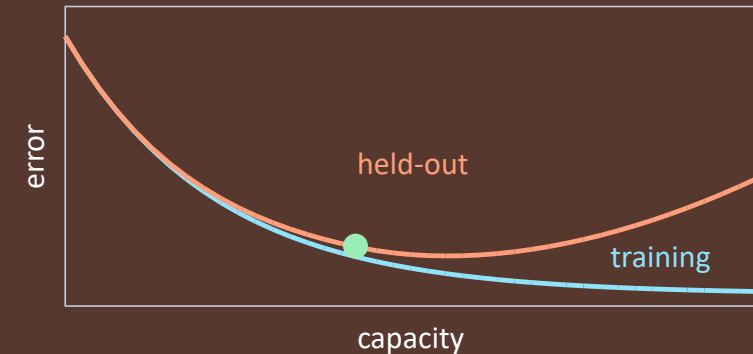


Model Capacity

HOW MANY FUNCTIONS A MODEL CAN REPRESENT

- **capacity** — the set of functions a model can represent; the parameter count sets it
- **the aim** — a model that represents the function behind the data, not every function
- **too little** — it cannot follow the truth; too much — it follows the noise
- **training error** — falls with capacity without limit
- **held-out error** — falls, then rises; the minimum is invisible from the training error

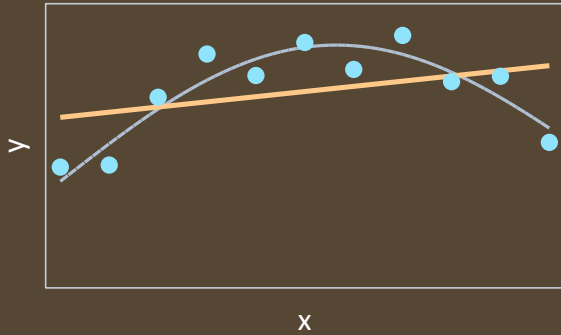
ERROR AGAINST CAPACITY



The green point is the capacity to aim for. It is found only with data held out of training.

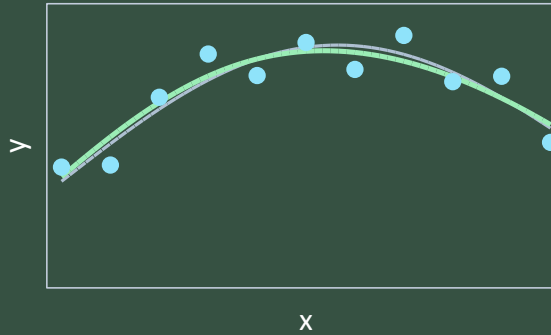
Underfitting and Overfitting

UNDERFITTING — TWO PARAMETERS



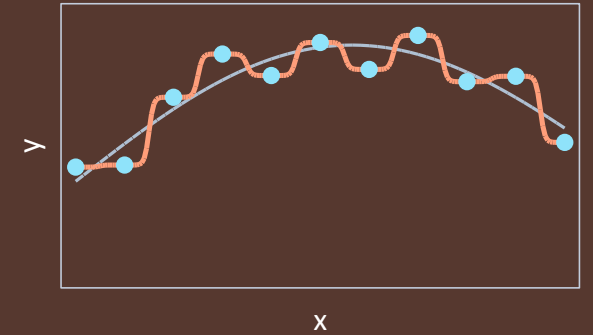
training error high, held-out high

MATCHED CAPACITY



training error moderate, held-out lowest

OVERFITTING — REACHES EVERY POINT



training error ≈ 0 , held-out worst

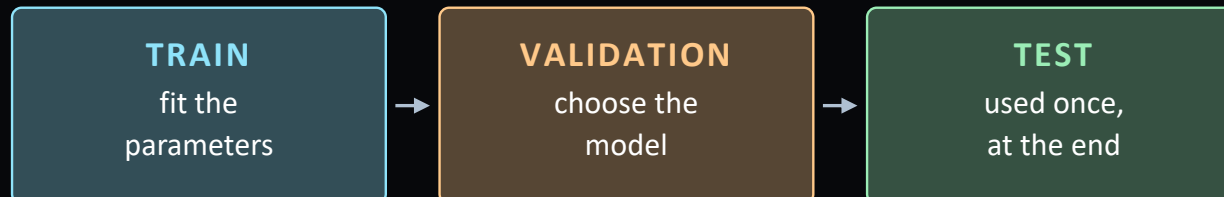
- **eleven noisy points** — the dashed curve is the truth, never available
- **underfitting announces itself** — the loss stays high
- **overfitting does not** — the training loss looks excellent

TALK · 2 MIN Look at the third plot. You never have the dashed curve. What measurement would tell you that this fit, with its perfect training loss, is the wrong one?

Data Splits

TRAINING, VALIDATION AND TEST

- **training set** — fits the parameters
- **validation set** — chooses between models: width, depth, when to stop
- **test set** — estimates the error on data nobody used to decide anything
- **used once** — a model chosen on the test set reports an error nobody can reproduce



HOW TO SPLIT

independent	at random
time series	by time, never at random
repeated runs	by run, not by sample
one rig, one batch	less data than it looks
small N	cross-validate

Two samples from the same test run are not independent. Split them at random and the training set contains the answer: the score is excellent and the model has learned nothing.

Bias and Variance

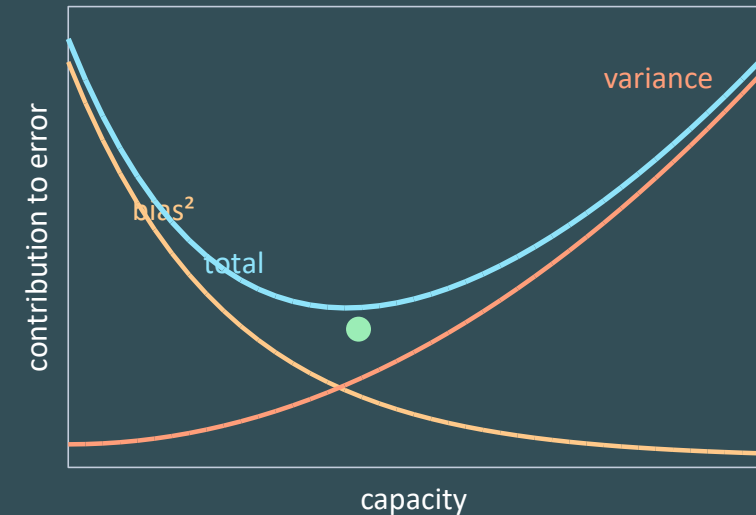
THREE PARTS OF THE ERROR

- **repeat the experiment** — new data, new noise, refit each time
- **bias** — how far the average fitted model sits from the truth
- **variance** — how much the fitted model moves when the data changes
- **noise** — the floor no model removes
- **an error below the noise floor** — suspect leakage before believing it

$$\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \sigma^2$$

y the truth, $\hat{f}$ the fitted model, $\mathbb{E}$ averaged over refits, σ^2 the noise floor

BIAS AND VARIANCE AGAINST CAPACITY



The classical picture: a U, with a best capacity in the middle. It holds for models with fewer parameters than data points.

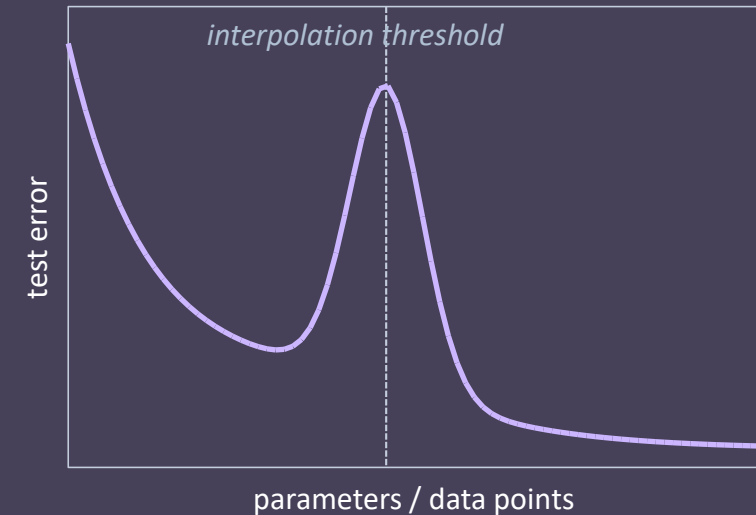
Double Descent

PAST THE INTERPOLATION THRESHOLD

- **the U** — holds for models with fewer parameters than data points
- **past the peak** — with many more parameters, test error falls again
- **the peak** — where the parameters can just fit the training set
- **why** — among the perfect fits, descent tends to find a smooth one
- **in practice** — with a few hundred samples you are left of the peak

TALK · 1 MIN On the purple curve, where would a network with 400 training points and 1,185 parameters sit, and which side of the dashed line does the classical U describe?

TEST ERROR AGAINST CAPACITY



The dashed line is where the model can just fit the training set exactly. Left of it, the classical U; right of it, the second descent.

Regularisation

FIVE METHODS, AND WHAT EACH DOES

- **weight decay** — add the summed squared parameters to the loss; large weights become costly
- **early stopping** — stop when the validation loss turns; the next slide

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_j \theta_j^2$$

$\mathcal{L}$ the data loss, $\tilde{\mathcal{L}}$ the regularised loss, θ_j each parameter, λ how much you care

- **dropout** — switch off random neurons during training, so none relies on another
- **data augmentation** — transformations that should not change the answer
- **a smaller model** — the method nobody lists

WHERE EACH IS USED

weight decay	Ex_04 notebook 05
early stopping	every training run
dropout	large classifiers; rarely here
augmentation	camera images, L11
a smaller model	the usual answer here
physics in the loss	Part 2 — the strongest of all

A physics residual is a regulariser: it penalises every violation of a conservation law, everywhere it is evaluated. Weight decay only discourages large weights.

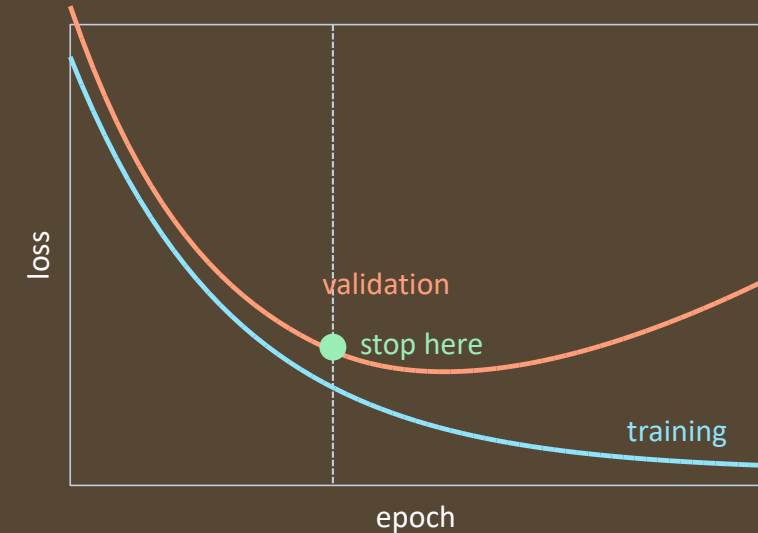
Early Stopping

TRAINING AND VALIDATION LOSS, ON ONE PLOT

- **plot both losses against epoch** — on the same axes, every run
- **both fall at first** — the model is learning structure
- **validation turns up** — the model is learning the noise
- **early stopping** — keep the lowest-validation checkpoint
- **read the shape** — flat and high: underfit; widening gap: overfit

PREDICT • 1 MIN On the plot to the right, the validation loss turns up at the dashed line. What do you keep from the run, and what do you change before the next one?

TRAINING AND VALIDATION LOSS



Right of the dashed line the model is memorising the training set. Keep the checkpoint from the green point.

Choosing Capacity

A PROCEDURE, NOT A FORMULA

- **start too small** — underfitting says so, and gives a floor
- **grow** — until the training loss is low; then read the validation loss
- **regularise before shrinking** — weight decay and early stopping
- **search at random, not on a grid** — grids waste trials

WITHOUT LABELLED DATA

A network trained on a physical law has no samples to hold out. Hold out conditions instead, check the residual at points not trained on, and check the conservation laws.

THE ORDER TO WORK IN

- 1 overfit five samples: is it wired up?
- 2 start small; note the underfit floor
- 3 grow until the training loss is low
- 4 plot training and validation together
- 5 add weight decay and early stopping
- 6 only then change the architecture

Step one belongs at the top of every procedure: a model that cannot memorise five points has a bug, and no architecture search fixes a bug.

Challenges

THE LOSS STALLS HIGH AND FLAT

underfitting or vanishing gradient · shrink the network: same loss means the gradient · He initialisation (2015) · ReLU

VALIDATION TURNS UP EARLY

overfitting, sooner than expected on small data · weight decay · early stopping (Prechelt 1998) · then the architecture

VALIDATION BETTER THAN TRAINING

leakage · the same run in both splits · normalised before splitting · split by run, by time (Kaufman et al. 2012)

DATA AND ENERGY EFFICIENCY

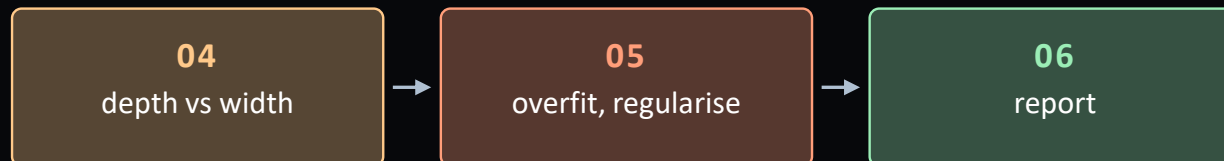
print	the parameter count, before training
plot	training and validation together
overfit	five samples on purpose, as a code test
halve	the width, and refit
check	how the data was split
collect	more data — most often right

Glorot & Bengio (2010), He et al. (2015) on initialisation; Prechelt (1998) on early stopping; GBC ch. 11 on practical methodology.

Exercise 4

NOTEBOOKS 04 TO 06

- **04** — depth against width: fix the parameter budget, spend it three ways
- **05** — overfit the eleven points of this lecture, then regularise
- **06** — the report: choose a model and defend the choice
- **every run** — training and validation loss on one plot



THE QUESTION EACH NOTEBOOK ANSWERS

- 04** at equal parameter count, deeper or wider — and does it depend on the problem?
- 05** what does weight decay recover from an overfitted model, and what does early stopping?
- 06** which model would you deploy, and on what evidence?

REPORT QUESTION

Which model would you deploy, and what would have to be true of the data for that choice to be right?

Key Takeaways

1 Depth buys efficiency, not capability

One layer can represent anything; several represent it with fewer parameters, because the regions multiply. Notebook 04 measures it.

2 Capacity is chosen, not maximised

Too little and the model cannot follow the truth; too much and it follows the noise. Notebook 05, first half.

3 Training error is not evidence

The overfitted model had the best training loss and the worst held-out error. Hold data back and plot both curves. Notebook 05.

4 Regularise before you shrink

Weight decay and early stopping recover most of what an over-large model loses. Notebook 05, second half.

5 The classical U is not the whole story

Past the interpolation threshold the test error can fall again; with a few hundred samples you are left of it. Notebook 06.

Next — L5.1: convolutional and graph networks.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What is a deep network, written as a composition, and what does one layer compute in matrix form? *UDL ch. 4.1 – 4.4*
- 2 Why does doubling the width cost four times the parameters while doubling the depth costs two? *UDL ch. 4.4; Ex_04 notebook 04*
- 3 Why do the linear regions multiply rather than add when layers are composed, and what does that say for depth? *UDL ch. 4.5*
- 4 What is the vanishing gradient, and which three changes made deep networks trainable? *UDL ch. 7.5, 11*
- 5 What is model capacity, and how do training and held-out error behave as it grows? *UDL ch. 8.1*
- 6 A model passes through every training point. Why is that not good news, and how would you detect it? *UDL ch. 8.1; Ex_04 notebook 05*
- 7 What are the jobs of the training, validation and test sets, and why is the test set used once? *UDL ch. 8.1*
- 8 What are the three terms in the bias–variance decomposition, and what does an error below the noise floor suggest? *UDL ch. 8.2*
- 9 What does double descent add to the classical U-curve, and when does it matter in practice? *UDL ch. 8.4*
- 10 Given an overfitted model, why regularise before shrinking it, and which two methods would you try first? *UDL ch. 9.1; Ex_04 notebook 05*

NEXT

L5.1 — Convolutional and Graph Networks

So far every layer has connected everything to everything. Next: one learned local rule applied everywhere, on a grid and then on a graph.

BEFORE THE NEXT SESSION

read	UDL ch. 10.1 – 10.2
run	Ex_04 notebooks 04 and 05
bring	the two loss curves from notebook 05
think about	why a dense layer on an image is wasteful

Bring the curve. L5.1 opens by asking three people to describe the shape of theirs.